is defined on sequences, this optimization relies on a mild condition that $\mathcal{T}$ is prefix-free, namely no target sequence $\mathbf{t}$ should be the prefix of another target sequence $\mathbf{t}'$. This condition is necessary since we would otherwise always have $\mathbf{p}_\theta(\mathbf{t}' \mid \mathbf{x}) < \mathbf{p}_\theta(\mathbf{t} \mid \mathbf{x})$, and it would be impossible to, say, make a model output identical probability for $\mathbf{t}$ and $\mathbf{t}'$.[3] Notice that minimizing the objective in Eq. 5 is equivalent to minimizing the KL-divergence between the model and the ground truth distribution $\mathbf{p}^\star$:

$$\begin{aligned}
\mathcal{L}(\mathbf{p}_\theta) &= \sum_{\mathbf{y}\in\mathcal{T}} \mathbf{p}^\star(\mathbf{y} \mid \mathbf{x}) \left(\log \frac{\mathbf{p}^\star(\mathbf{y} \mid \mathbf{x})}{\mathbf{p}_\theta(\mathbf{y} \mid \mathbf{x})} - \log \mathbf{p}^\star(\mathbf{y} \mid \mathbf{x}) \right) \\
&= \sum_{\mathbf{y}\in\mathcal{T}} \mathbf{p}^\star(\mathbf{y} \mid \mathbf{x}) \log \left(\frac{\mathbf{p}^\star(\mathbf{y} \mid \mathbf{x})}{\mathbf{p}_\theta(\mathbf{y} \mid \mathbf{x})} \right) - \sum_{\mathbf{y}\in\mathcal{T}} \mathbf{p}^\star(\mathbf{y} \mid \mathbf{x}) \log \mathbf{p}^\star(\mathbf{y} \mid \mathbf{x}) \\
&= D_{\mathrm{KL}}\left(\mathbf{p}^\star \parallel \mathbf{p}_\theta\right) - \mathrm{Entropy}(\mathbf{p}^\star),
\end{aligned}$$

where the entropy term is independent from the model distribution. In this view, our method can be seen as *distribution matching*, where we align the model distribution with an ideal distribution.

In the more complex case where the ground truth distribution is not known, our method assumes access to a small sample set ($\sim$200 samples) and treats the empirical distribution as a proxy of ground truth. While minimizing this loss to 0 could lead to a model that exclusively produces targets from the sample set, which is not what we want in a case like synthetic dataset generation, empirically, we find this not to be the case. We discuss results on generalization beyond training distribution extensively in Section 5.

3.2 Parameter-efficient Fine-tuning

We use LoRA (Hu et al., 2021) to fine-tune models in a parameter-efficient way. This technique relies on optimizing low-rank additions to weight matrices, making fine-tuning large language models (with $\geq$ 7B parameters) practical. It has the added benefit that LoRA tuning tends to perform almost as well as full model fine-tuning and generalizes beyond training instances (Malladi et al., 2023). From an implementational standpoint, LoRA requires instantiating a new pair of relatively small matrices for every weight matrix in the network. Consider one fully connected layer parameterized by $W \in \mathbb{R}^{d\times d}$. We instantiate $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ where $r \ll d$ and reparametrize the layer as $W + BA$. When fine-tuning the model, we exclusively optimize the weights of A and B rather than taking gradient steps on W, enabling us to fine-tune large models such as Llama-2-13B with only a small fraction of the memory overhead.[4] We report training details in Appendix A.

4 Results on Simple Tasks

We begin by showing how state-of-the-art language models fail to produce diffuse distributions on simple tasks, even when a user's prompt requests randomness. We consider two tasks: random baby name generation (BABYNAMES) and random number generation (RNG). In BABYNAMES, the model is instructed to generate a random baby name in English. In RNG, we ask the model to generate a random number between 1 and 10. The tasks have the following prompts:

1. BABYNAMES: "Please generate an English first name, chosen completely at random. Output only the name between two curly braces, like this: {name}. Don't output code."
2. RNG: "Generate a random number between 1 and 10. Output only the number between two curly braces, like this: {number}. Don't output code."[5]

[3] We can simply append an end-of-sequence token to each target if $\mathcal{T}$ is not already prefix-free.

[4] Empirically, we find that $r = 4$ is sufficiently expressive, which we use across all experiments. On our models, this corresponds to tuning less than 0.1% of all model parameters.

[5] We instruct the models to generate between curly braces to simplify output parsing. Gemma has a strong tendency to output code and we add an explicit instruction to avoid code generation.

Table 1: Our method significantly increases the diversity of generations on the random number and baby name generation tasks. Coverage-1000 is the number of unique outcomes out of 1,000 trials.

Task	Model	Entropy ↑			Coverage-1000 ↑		
		Baseline	Tuned	Natural	Baseline	Tuned	Natural
RNG	Gemma	1.16	2.28	2.30	8	10	10
	Llama-2	1.07	2.29		7	10	
	Mistral	1.88	2.30		9	10	
BABYNAMES	Gemma	3.19	5.69	6.43	170	442	714
	Llama-2	3.24	5.83		129	490	
	Mistral	4.02	5.78		225	479	

We consider three state-of-the-art language models: Gemma-7B-instruct (Google, 2024), Llama-2-13B-chat (Touvron et al., 2023) and Mistral-7B-instruct (Jiang et al., 2023). We use chat/instruct versions of the models, which are fine-tuned on dialog data to make them more suitable than base models for our test cases, which require instruction following. In all experiments, we sample from models using a temperature of one, which is equivalent to sampling from the maximally diffuse, unmodified model distributions.[6] For each task, we sample 1,000 generations from the model and report coverage and entropy statistics of the empirical distributions of model generations.

4.1 Language models fail to follow instructions for randomness

Figure 1 shows output distributions of several untuned, baseline models for BABYNAMES and RNG. Despite the prompts asking for randomness, these output distributions are extremely imbalanced.

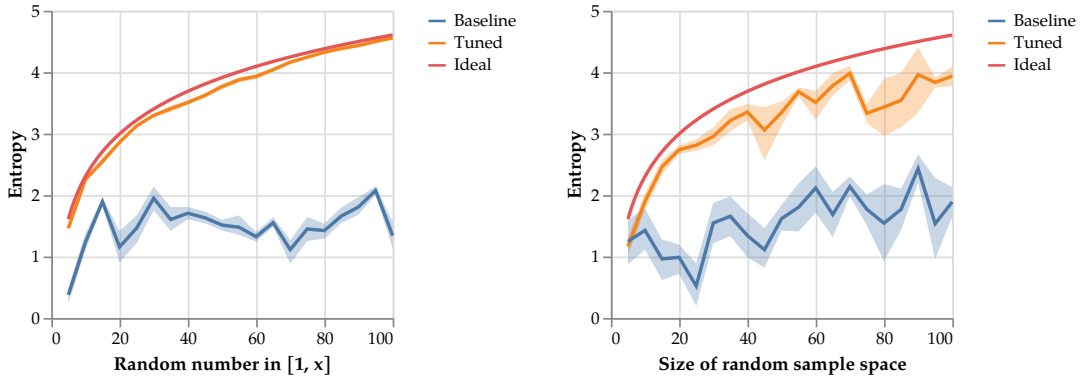
For RNG, it is easy to see that the empirical distribution is far from the uniform distribution we would expect. For BABYNAMES, it is less clear what we should expect from the model. Do we want the generated names to be a uniform sample from the set of all names? Or do we want the names to reflect the distribution of baby names among recently born babies in the Unites States? To address this, we introduce the concept of a *natural* distribution for each task. For RNG, this is simply the uniform distribution; for BABYNAMES, we compare to a sample we draw from data on names used for babies born in the United States in the year 2022, using birth data from the US Social Security Administration.[7] We note, that we are only using the natural distributions as reasonable baselines for comparison.

When generating names, we find that Llama-2 generates the name "Aurora" about 100 times more often than we would expect from the natural distribution, and Gemma assigns 40% probability to the name "Anya", which did not appear even once in our sample of the natural distribution. This is a clear indication that these large language models do not sample diversely from a large space of valid generations (all English names) despite being explicitly instructed to do so.

In the RNG task, we find that each model has a bias toward a different set of numbers. For example, both Gemma and Llama-2 assign over 60% probability to the number five, while Mistral assigned over 25% probability to the number eight. Surprisingly, in 1000 samples, none of the models were able to generate all ten numbers. These results highlight that the output distributions of large language models are far from diffuse, even in simple tasks where a user may expect them to be.

[6] We could technically set temperature to beyond one to smooth out the model distribution further, but this is rarely done in practice due to degeneration. We also note that the untuned, baseline models need to be sampled at a temperature > 10 to match the entropy of our fine-tuned models in Section 4.2.

[7] https://www.ssa.gov/OACT/babynames



(a) Varying prompt formats and number ranges.

(b) Varying sizes of random sample space.

Figure 2: Models tuned on RANDOM NUMBER GENERATION demonstrate generalization to variations in both prompt format and number ranges. 95% confidence intervals are shown in plots.

4.2 Optimizing for BABYNAMES and RNG diversifies generation

We start by focusing on the setting where we are optimizing the model to produce diffuse probabilities for the two individual tasks. For RNG, we fine-tune the model to produce a uniform distribution from one to ten, and for BABYNAMES, we fine-tune the model to fit the empirical distribution of a sample of 200 baby names drawn from the natural distribution.

In these experiments, we see our methods can very effectively align the model distribution with the ideal distributions (Table 1). With RNG, tuned models produce near-uniform distributions over the valid choices with full coverage and near perfect entropy. With BABYNAMES, where the training set covers only a small portion of the possible outputs, we see an encouraging indication of generalization: while the coverage remains smaller than that of the natural distribution, it is over twice the baseline for all models and the majority of outputs are not among targets in the fine-tuning set.

While RNG and BABYNAMES serve as toy problems, these settings have a major limitation—the fine-tuned models are evaluated on the same prompts and sample spaces in training. To further test the generalization performance of our models, we also consider settings where the prompt and sample space differ between training and evaluation. We fine-tune Llama-2 to produce diffuse probabilities over two ranges of random integers—one to ten and one to 100—and test its generalization to different instruction formats and unseen sample spaces (see Appendix B.1 for prompts). In Figure 2a, we observe encouraging generalization trends: the tuned model produces near uniform distributions for unseen prompt formats over number ranges not in the fine-tuning set, for example from 1 to 45. When we vary the size of the random sample space of RNG (e.g., 154 to 204), the tuned model still produces substantially higher entropy distributions than the untuned model (Figure 2b). With these first experiments, we demonstrate that tuning models for diffuse probabilities is a promising method for increasing generation diversity.

5 Generalization Across Tasks

It is not too surprising that a model optimized to output random numbers uniformly across some range can generalize to other ranges of numbers. A much more interesting and practical test is if a model optimized for diffuse probabilities on one set of tasks can transfer to tasks with very different sample spaces. That is, a model optimized for picking a random number or baby names should not overfit to generating samples from those distributions; rather, it should be able to e.g. pick a random fruit or country name when prompted to do so.